Instructions

Intro to R: Basics, Vectors, Matrices, Data Frames, and Lists

GOVT16/QSS17: Data Visualization (Prof. Cooper)

Week 1: Introduction to R

- Dates: March 31st - April 7th, 2026
- Time: Due Tuesday, April 7th, 11:59 PM to Canvas

Preparation

1. During this term, you should consider using a specific folder within your computer – your "working directory" – where you will save your R scripts, data, figures, and other documents. To make this folder, open RStudio and go to"File" > "New Project". Then, choose "New Directory", and make a new "project" (i.e., a new directory) somewhere on your computer. The project name, which is the name of your working directory, can be anything. Check that the new folder is generated with a new `.Rproj` file. As long as all of your files for this course are saved in this working directory, you need to make this `.Rproj` file once. When you work on your own project after this course, however, you should specify a different working directory with a new `.Rproj` file for each project.

What should you do in Week 1?

- In RStudio, choose "File" > "New File" > "R Script". Then, save the file as "Week01_script.R".
- Work on Exercises below.
- You should upload your R script by Tuesday, April 7th, 11:59 pm to Canvas ("Assignments" > "Week 1 Assignment").

Notes:

- Normally I would ask you to not write or type your answers *within your script*. But feel free to write some comments (with `#`) for your own memo. At this stage I encourage you to write reasonably extensive comments in your own version of your scripts so that you remember what it is you did and why you did it. Scripts to be read by others, under normal circumstances, should not be quite so heavyhanded with comments.

- Never use `T` and `F` as synonyms for `TRUE` and `FALSE`, because `T` and `F` can be redefined and could cause confusion and error. See `https://www.r-bloggers.com/r-tip-avoid-using-t-and-f-as-synonyms-for-true-and-false/`

- Each time I ask you to create a new object, please follow the creation of that object by printing the object. Under normal circumstances you would not print out every object in your R script, whether using proper workflow on your own or sharing scripts with other researchers and collaborators. It will be useful for all of us *in the context of this course*. You can check your work constantly by seeing the output and I can grade more efficiently by seeing the output.

- If I use a particular verb or reference a particular operator or function that you do not immediately recognize, there are many resources out there to help you to discover the solution. The process of search and discovery for basic coding solutions is a part of programming, and **R** has many online resources for you. Google is your friend, as is any one of a number of sources for **R** support.

Exercises

Set up the top of your R script with your name, the name of the course, the term, and the date using comments. There are different style guides for comments. For your homeworks, please use `#` (followed by one space) to add a comment to your code. Begin with `##` (followed by one space) for entirely commented lines.

```
> ## Data Visualization (GOVT16-QSS17) Spring 2026
> ## Intro to R, Part 1 & 2
> ##
> ## Name: YOUR NAME GOES HERE
> ## Date: March 31- April 7, 2026
```

Objects, Vectors, and Matrices

1. Objects.

   (a) Create an object called "Harry" and assign in the value 50.

   (b) Create an object called "Hermione" and assign it the value 47.

   (c) Create an object called "Ron" and assign it the value 44.

   (d) Create an object called "Ginny" and assign it the value 40.

   (e) Show me proof that the Harry and Ginny together are not greater than Ron and Hermione with a logical operator. The "answer" here is not a number, but instead a `TRUE/FALSE` or a "logical" vector.

   (f) Create an object called "Snape" that takes the value 49.

   (g) Create an object called "Voldemort" that is the inverse of "Harry." (e.g. -$\frac{1}{4}$ is the inverse of 4 because the product of $\frac{1}{4}$ and 4 is 1)

   (h) Prove that Harry and Voldemort are inverses of each other. Your answer, of course, should be 1. *Note: because of floating point rounding error, some numbers may look incredibly close to 1, as opposed to 1, due to computational rounding.*

2. Let's do some basic math in R without creating objects. Order of operations is key.

   (a) Calculate $7546 + 9918$.

   (b) Calculate $3467 - 8493$.

   (c) Calculate $37 \times 47 + 30$.

   (d) Calculate $(3 + 67) \times (8 - 29)$.

   (e) Calculate $656 + 23/53$.

   (f) Calculate 74^2.

(g) Calculate the remainder of 999/77 using the modulo operator. (If you do not know the modulo operator, it is easy to look up. It has to do with remainders.)

3. (Atomic) Vectors

   (a) Let's take Harry and Hermione and make a vector called "wizards1."

   (b) Let's take Ginny and Ron and make a vector called "wizards2."

   (c) Let's attach `wizard2` to the end of `wizard1`, making a longer vector called `wizards`.

   (d) Let's add labels/names to the vector, attaching each wizard's name to their wizard rating.

   (e) Remember when I said make Snape and Voldemort? On second thought, Snape and Valdemort should go away. Please `remove` those objects from R's memory.

   (f) We'd better be sure that Snape and Voldemort are gone; type each of those objects individually and prove it with an error message. You can use the `print` function if you want to.

   (g) Extract the value for Ginny from `wizards` using brackets.

   (h) Extract the values for Hermione and Ron from `wizards` using one command.

   (i) Extract the values for everyone but Harry. Use a colon in your subsetting this time.

   (j) Use a logical operator to show whose scores are lower than 45. *Hint*: the answer should be a series of non-numbers.

4. Matrices

   (a) Start by creating a vector named `scores` using the following numbers: {75, 49, 68, 83, 97, 91, 98, 89, 91}.

   (b) Turn `scores` into a matrix with the `matrix()` command. Make it have 3 rows and 3 columns. Call it `wiz_scores`. Feed the vector into the matrix *by row*.

   (c) These numbers represent the grades of Ron, Harry, and Hermione, in that order. Label the rows of the matrix with their names (Ron should be row 1) and the columns with `test1`, `test2`, and `test3`.

   (d) Make a new matrix object, `wiz_scores2`, using the `wiz_scores` matrix and binding it to a new column that represents mean grades for each of the wizards. Don't forget to add a column name `avg` for the averages column. *Hint*: a similar command to `ColSums` can be used here to compute the averages.

(e) It looks like Neville and Ginny also took the tests and had the following scores: 81, 80, 78, 92, 87, 84. Neville's scores are the first three, and Ginny's are the last three scores. Make a matrix called `otherwiz` for their scores and set it up the same as you did for the other wizard students, including averages. Label with wizard names on rows and `test1`, `test2`, `test3`, and `avg` over the columns.

(f) Bind the scores for Neville and Ginny to the scores for the other wizards. Bind the matrices using `rbind`. Call the full matrix `all_wiz_scores`.

(g) Extract Hermione's second and third test scores using the matrix subsetting operator (hard brackets). Just print/show them. You should be able to see both the exam label and the score.

(h) Oops! `test2` had a flaw! Make a matrix `allwiz_sub` that subsets `all_wiz_scores` by eliminating `test2` from the matrix. Recalculate the averages, replacing `avg` with the `newavg` in the column names.

(i) Neville's `test3` was graded incorrectly! Go in and change the score to 98 by recoding that one element in the matrix. One more time, recalculate the averages. Replace the average scores and relabel the `newavg` as `finalavg`. We are now done with the wizards.

Factors, Data Frames, and Lists

1. **Categorical Variables: Characters & Factors**

(a) Create an object called `truth` that takes the value "Dartmouth is, like, way better than all the other schools. Like, it's not even close, you guys."

(b) Check the `class` of `truth` and print/report it.

(c) Create a vector called `colors` that takes the following values: red, blue, green, red, and blue. Put the values in quotations inside the vector.

(d) Create a factor vector called `factor_colors` from the `colors` vector.

(e) Create another factor called `factor2_colors` from the `colors` vector. This time set the argument `levels` to just red and blue. The result should be different from the previous factor vector.

i. What does this tell you about factors? Answer briefly in a comment.

ii. You are going to need to know about the properties of factors in this class and pretty much all your future research!

(f) Report summaries (using the `summary` function) of `colors`, `factor_colors`, and `factor2_colors`.

i. Note the differences between the summaries of characters and factors. Note also the differences between two factors of the same variable but with different levels.

(g) Create a vector called `ideology` that take the following 10 values: "liberal," "conservative," "very liberal," "very conservative," "middle of the road," "slightly conservative," "slightly liberal," "liberal", "conservative," "middle of the road."

(h) Now create a factor called `fact_ideo` from the `ideology` vector. Make sure that the levels of `fact_ideo` are *ordered* appropriately, from "very liberal" to "very conservative." *Note: There is more than one way to reorder a factor.*

(i) Create a character vector called `respondents` that takes the following 10 values: Susie, Abdul, Maria, Fred, Wilma, Barney, Dino, Ajax, Thor, and Betty. Put them all in quotations in the vector. This will be used later.

2. **Data Frames**

(a) Create a vector called `income` that takes the following values: 100000, 75000, 48000, 62000, 31000, 52500, 274000, 88000, 21000, and 74000. Don't add commas inside the numerical values; **R** will incorrectly read the values as characters if you do.

(b) Create a data frame from your `respondents`, `ideology`, and `income` vectors/variables, calling it `data1`.

(c) Show me the head of `data1`.

(d) Show me the tail of `data1`.

(e) Show me structure of `data1`.

(f) Order the observations in the dataset by `income` in descending order (highest first), making a new copy of the data and calling it `orderdat`.

(g) Go back to `data1` and extract from `data1` the respondent with the lowest income; subset/isolate the entire row. Pull the row out and print it.

(h) Select from `data1` the names of all the respondents starting with Fred and ending with Ajax. There are a number of ways to do this. *Note*: Do not reorder the names alphabetically. I would like the subset of rows in the order they sit in the data frame.

(i) There is at least one large outlier in the sample. Add a new variable to `data1` called `log_income` that takes the natural log of the `income` variable.

3. **Lists**

(a) Take your `ideology`, `respondents`, and `income` variables and put them into a list object called `survey`. See how this structure differs from when you created `data1`.

(b) Create an object `session` that takes the value 2. Just a single number.

(c) Create an object `weeks` that is a 3×3 matrix of numbers 1 through 9. Input them `byrow`.

(d) Create a list called `dv_list` that takes `truth`, `session`, `weeks`, your dataset `data1`, and the `survey` list object. Rename these objects `truth`, `sess1`, `wk`,`dat`,and `svy` inside the `list` command.

(e) Extract the middle element from the `weeks` matrix *from* ***`dv_list`***. The code to execute this should thus start with the list itself.

(f) Change the `income` variable in `data1` *from within* ***`dv_list`***. Divide `income` by 2. The code to execute this should start with the list object itself.